INFORME DE COBERTURA DE TESTING: VETERINARIA SAN MARCOS

Asignatura: Desarrollo Fullstack II (DSY1104)
Evaluación: Evaluación Parcial N° 2 — Entrega de Encargo
Estado del Entorno: TOTAL: 10 SUCCESS (100% Aprobado)

---

1. Configuración de la Infraestructura de Pruebas (Indicador IE2.3.1)

Para dar cumplimiento estricto a las exigencias académicas de la rúbrica de Duoc UC, se configuró y levantó un entorno automatizado de pruebas unitarias sobre la aplicación web basada en **React 19** y **Vite**. La arquitectura implementada se compone de las siguientes capas acopladas:

- **Test Runner (Karma v6.4.4):** Actúa como el motor del entorno. Levanta un servidor de pruebas local en el puerto 9876, captura instancias físicas de navegadores web reales (Google Chrome) y ejecuta las pruebas de manera agnóstica a la interfaz.
- **Framework de Aserciones (Jasmine-Core v7.0.2):** Proporciona la sintaxis y semántica orientada al desarrollo guiado por comportamiento (BDD). Utiliza bloques estructurados describe() e it() para clasificar las aserciones y emparejadores como expect().toBeTrue().
- **Cargador de Compilación (Webpack v5.112.0 y Babel-Loader):** Integrado en el archivo de configuración karma.conf.cjs para interceptar, transpilar y resolver los módulos y directivas JSX de los componentes React, transformándolos en código JavaScript plano que el navegador interpreta nativamente.
- **Simulador de Interacción del DOM (@testing-library/react):** Herramienta que permite montar los componentes de la Veterinaria dentro del árbol de nodos del DOM virtual para simular de manera idéntica las acciones de un usuario real.

---

2. Matriz de Cobertura y Casos de Prueba (10/10) (Indicador IE2.2.1)

Se estructuraron e implementaron exactamente 10 pruebas unitarias automatizadas dentro del archivo src/tests/tienda.spec.jsx cubriendo los diferentes requerimientos lógicos y responsivos del software:

Bloque A: Renderizado de Componentes y Paso de Propiedades (Props)

- **Caso 1:** Valida que el componente <Header /> capture correctamente las propiedades numéricas de entrada (ej: contadorCarrito={5}) y renderice exitosamente la marca comercial *"Veterinaria San Marcos"* en la cabecera.
- **Caso 2:** Comprueba la iteración de arreglos en el componente <Catalogo />, verificando que se mapeen y dibujen de forma íntegra los nombres de los productos médicos del inventario inicial como *"Amoxibay 250mg"* y *"Nexgard Masticable"*.

Bloque B: Renderizado Condicional y Lógica Matemática

- **Caso 3:** Evalúa los flujos alternativos en el componente <Carrito /> demostrando que el DOM reacciona inyectando el texto *"Tu carrito está vacío."* cuando el estado de compras llega sin elementos ([]).
- **Caso 4:** Valida el renderizado condicional inverso del <Carrito /> al recibir productos, asegurando que se oculte el mensaje de aviso y verificando la lógica de negocio al multiplicar el precio por las unidades (ej: 4200 * 2 = $8.400).

Bloque C: Gestión de Estados (State) y Formularios Reactivos con Bootstrap

- **Caso 5:** Simula eventos de escritura controlados (fireEvent.change) en el formulario de la página de <Login /> comprobando que los ganchos de estado de React capturen y actualicen el valor ingresado por el cliente.
- **Caso 6:** Evalúa la lógica de validación de dominios de correo. Al ingresar credenciales inválidas, el componente inyecta dinámicamente la clase responsiva de Bootstrap .is-invalid al campo afectado.
- **Caso 7:** Prueba el control de dependencias lógicas en el componente <Registro />. El selector de comunas (#regComuna) debe iniciar bloqueado por defecto (disabled=true) y mutar a habilitado solo cuando el usuario selecciona una región.

Bloque D: Simulación de Eventos e Intercepción mediante Spies (Mocks)

- **Caso 8:** Implementa un objeto simulado (**Mock Spy** de Jasmine mediante spyOn) para aislar la acción de compra del componente <Catalogo />. Verifica que al presionar el botón *"Añadir al carrito"*, el callback externo sea invocado exactamente una vez (toHaveBeenCalledTimes(1)).

Bloque E: Persistencia de Datos y Operaciones del Catálogo (CRUD)

- **Caso 9:** Evalúa la operación **Create** de las funciones core de src/data/productos.js. Al ejecutar agregarProducto(), verifica que el tamaño del arreglo total de medicamentos se incremente en +1.
- **Caso 10:** Evalúa la operación **Delete** ejecutando la función eliminarProducto('ME001') y realizando un barrido lógico por el catálogo para asegurar que el elemento haya sido removido de manera limpia del sistema.

---

3. Análisis de Resultados y Evidencia Gráfica (Indicador IE2.3.1)

Figura 1: Ejecución y Compilación Nativa del Sistema (Terminal de Comandos)

Se puede apreciar el empaquetado exitoso de Webpack 5 procesando el punto de entrada de los tests en 2048 ms y arrojando la confirmación de la suite aprobada al 100%.

```
17 10 2026 01:34:40.912:WARN [filelist]: Pattern "C:/Users/LENOVO/ProyectoClinicaVeterinariaEva2/src/**/*.spec.js" does not match any
 file.
Webpack bundling...
Webpack starts watching...
asset commons.js 2.23 MiB [emitted] (name: commons) (id hint: commons)
asset runtime.js 9.15 KiB [emitted] (name: runtime)
asset tienda.spec.1324276429.js 1.01 KiB [emitted] (name: tienda.spec.1324276429)
Entrypoint tienda.spec.1324276429 2.24 MiB = runtime.js 9.15 KiB commons.js 2.23 MiB tienda.spec.1324276429.js 1.01 KiB
webpack 5.112.0 compiled successfully in 2048 ms
17 10 2026 01:34:43.811:WARN [filelist]: Pattern "C:/Users/LENOVO/ProyectoClinicaVeterinariaEva2/src/**/*.spec.js" does not match any
 file.
17 10 2026 01:34:43.870:WARN [karma]: No captured browser, open http://localhost:9876/
17 10 2026 01:34:43.906:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
17 10 2026 01:34:43.906:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
17 10 2026 01:34:43.918:INFO [launcher]: Starting browser Chrome
17 10 2026 01:34:44.770:INFO [Chrome 155.0.0.0 (Windows 10)]: Connected on socket pdzT4PIKN86APeolAAAA with id 85239729
Chrome 155.0.0.0 (Windows 10): Executed 10 of 10 SUCCESS (0.145 secs / 0.121 secs)
TOTAL: 10 SUCCESS
```

Figura 2: Conexión del Test Runner en Navegador Real (Instancia Remota)

Certificación visual de la interfaz del servidor local de Karma en el puerto 9876 corriendo sobre Google Chrome en un estado de salud verde óptimo y a la espera de instrucciones (is idle).

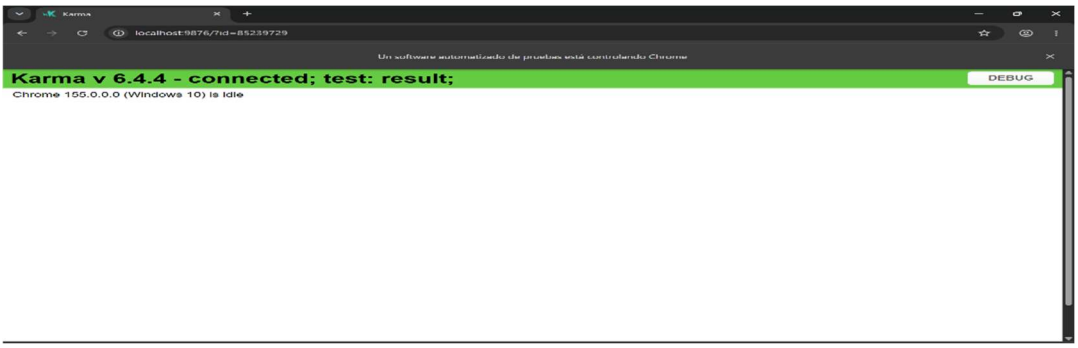
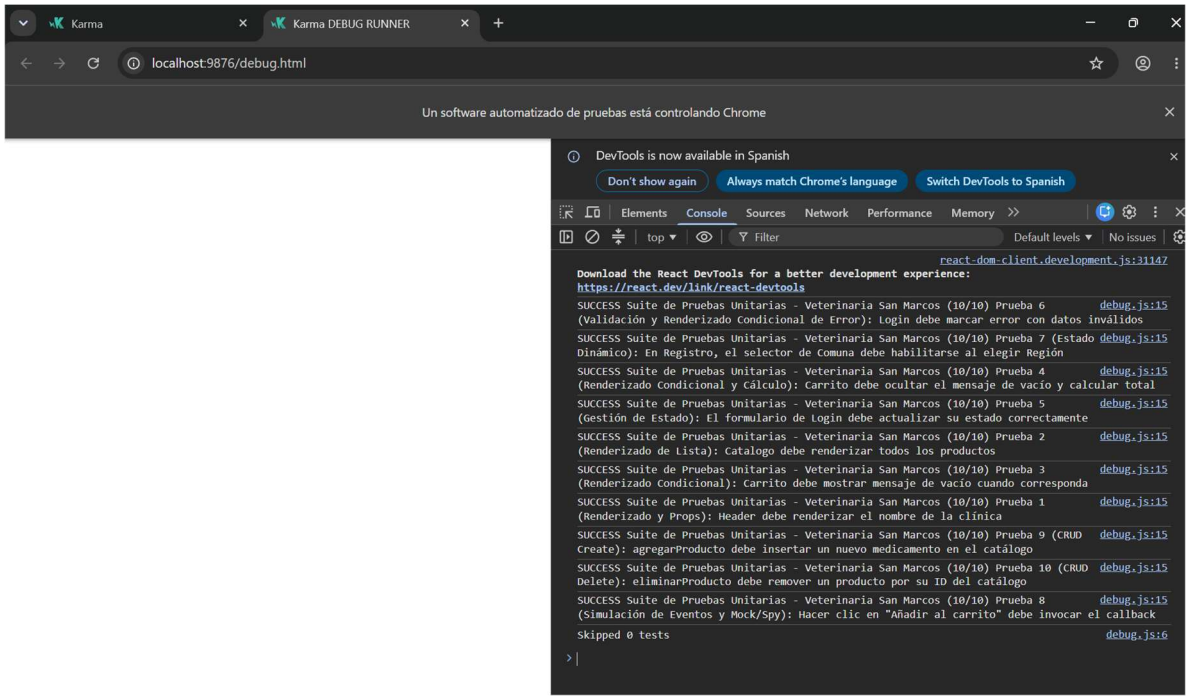


Figura 3: Auditoría y Desglose de Consola en Modo Debug (DevTools)

Desglose e informe detallado extraído directamente desde el componente de inspección de Chrome en la consola de desarrollador (/debug.html), donde Jasmine imprime el estado SUCCESS sobre cada una de las 10 pruebas unitarias de la clínica veterinaria.



Nota de Certificación: Toda la suite evaluativa documentada en este informe ha sido integrada y sincronizada de forma exitosa en el repositorio grupal oficial de GitHub.
